

**Project: Darknet Crawler (XE109) Student Name:** Yonatan Avitan **Student Code:** s20 **Class Code:** 7736-39 **Lecturer:** Zach Azolis

# Project Directive: Darknet Crawler Automation (Project Code: XE109)

## 1. Context & Objective (The "Description" Phase)

You are acting as an expert Senior Systems Architect and Cybersecurity Developer assisting me in completing a graded academic project titled "Darknet Crawler."

Our primary objective is to build a highly automated Bash script (`.sh`) that crawls the darknet (specifically `.onion` websites), extracts and indexes links, and manages data with minimal human intervention.

This project requires strict adherence to specific architectural steps, constraints, and submission guidelines. The final product must be a single, robust Bash script.

**\*\*Important Note on AI Assistance:\*\*** As part of the project requirements, I am permitted to use AI chatbots for guidance. However, the resulting code must be fully functional, well-documented, completely transparent, and uniquely mine. I am the driver; you are the navigator. Do not generate code using external, unverified libraries. Stick to standard Linux utilities (`curl`, `grep`, `awk`, `sed`, `wget`, `xargs`, etc.) and Tor/proxy configurations.

## 2. Project Requirements & Architecture

The script must be structured to fulfill the following requirements sequentially and comprehensively:

### Phase A: Environment Setup & Validation (Pre-flight checks)

1. **\*\*Network Validation:\*\*** The script must verify that the environment is configured to access the Tor network. It should check if `tor` service is active or if `proxchains` or `nipe` is configured correctly.

2. **Connectivity Test:** Attempt to connect to a known, stable `.onion` site (or check the Tor network status) to validate darknet access before proceeding. If access fails, the script must halt and instruct the user on how to resolve the issue.

### ### Phase B: User Interaction & Input (Module 1)

1. **Seed List Input:** Prompt the user to provide the path to a text file containing exactly 10 `.onion` websites to serve as the initial crawl seeds. Validate that the file exists and is not empty.
2. **Alert Keywords:** Prompt the user to input specific search words (comma-separated or individually) that will trigger alert notifications if found during the crawl.

### ### Phase C: The Crawling & Indexing Engine (Module 2)

1. **Iteration:** Iterate through each `.onion` URL provided in the seed list.
2. **Concurrency & Performance Optimization:** Maximize CPU core utilization to ensure high-performance crawling. Use parallel processing techniques appropriate for Bash (such as `xargs -P` or background jobs `&` with `wait`) to crawl multiple `.onion` sites simultaneously, significantly reducing execution time while maintaining stability.
3. **Data Extraction:** For each site, the script must extract the `<title>` of the webpage.
4. **Logging & Persistence:** Save the extracted data into a structured log file (e.g., `crawler_log.txt`). The log format must include: `[Date] [Time] [URL] [Title]`.
5. **Link Discovery (Indexing):** Extract all new `.onion` links found within the HTML of the crawled pages. Append these new links to an index file for future crawling.
6. **State Management & Resumption:** The crawler must track its progress. If the script is interrupted or restarted, it must read its state and resume crawling from the exact point it left off, skipping already crawled sites. Ensure state tracking is thread-safe if running concurrently.
7. **Deduplication:** Before crawling a URL or adding it to the index, verify that it has not been crawled previously. Check only the active status of known URLs, do not re-crawl and re-index their content.

### ### Phase D: Error Handling for Inaccessible Sites (Module 3)

1. **\*\*Timeout & Failure Handling:\*\*** If a `.onion` site is down, times out, or cannot be accessed, the script must gracefully handle the failure.
2. **\*\*Specific Log Requirement:\*\*** For inaccessible sites, the script *must* log the entry with the exact comment: `[NO ACCESS]`. Format: `[Date] [Time] [URL] [NO ACCESS]`.

### ### Phase E: Admin UI & Reporting (Module 4)

The script must feature an interactive menu-driven interface (the "Admin UI") that allows the user to monitor and manage the process. The UI must display:

1. **\*\*Crawl Statistics:\*\*** The total number of sites successfully crawled and indexed.
2. **\*\*Network Status:\*\*** The current darknet access status (displayed strictly as `Active` or `Non-Active`).
3. **\*\*Keyword Management:\*\*** Options to view current alert keywords, add new search words, or remove existing ones.
4. **\*\*Log Access:\*\*** An option to view the contents of the log file directly from the UI (e.g., using `cat`, `less`, or `tail`).

### ### Phase F: Academic Compliance & Metadata

The script must contain a specific header block containing comments with the following exact details:

- \* Student Name: Yonatan Avitan
- \* Student Code: s20
- \* Class Code: 7736-39
- \* Lecturer Name: Zach Azolis

## ## 3. Execution Strategy (The "Delegation" Phase)

Do not write the entire monolithic script in your first response. I want to build this systematically to ensure quality control and understandability.

**\*\*Step 1: Architecture & State Design:\*\*** First, respond with a proposed architecture. Detail how you plan to manage state (resumption), handle concurrency/parallelism safely, how you will extract HTML titles robustly using bash tools, and how the file structure (logs, indexes, seed lists) will be organized.

**\*\*Step 2: Core Engine:\*\*** Once I approve the architecture, provide the core crawling function (Phase C & D). We will test this module independently.

**\*\*Step 3: UI & Input:\*\*** Next, we will wrap the core engine in the Admin UI and handle user inputs (Phase B & E).

**\*\*Step 4: Integration & Polish:\*\*** Finally, we will integrate network validation and the academic metadata.

**\*Wait for my approval after Step 1 before writing any executable code.\***

#### **## 4. Quality Assurance & Evaluation (The "Discernment" Phase)**

When proposing your architecture and writing code, you must apply the following discernment criteria:

**\* \*\*Robustness over Complexity:\*\*** Bash is prone to errors with string manipulation. How will you ensure that extracting titles and links doesn't break if the HTML is malformed? Use robust tools (like ``grep -oP`` or ``sed``) and explain your regex choices.

**\* \*\*Security:\*\*** We are interacting with darknet traffic. Ensure the script does not accidentally leak DNS requests outside the Tor network. Mention how you will enforce proxy usage for commands like ``curl``.

**\* \*\*Hallucination Check:\*\*** Do not invent bash commands or assume the presence of third-party parsing libraries (like ``jq`` or ``pup``) unless absolutely necessary, and

if so, clearly state that they must be installed. Stick to POSIX-compliant or standard GNU tools as much as possible.

## ## 5. Professional Responsibility (The "Diligence" Phase)

The final deliverable is an academic project. Therefore:

- \* The code must be heavily commented, explaining *\*why\** a particular command or regex is used, not just *\*what\** it does.
- \* Variable names must be highly descriptive (e.g., ``TOTAL_SITES_CRAWLED`` instead of ``tsc``).
- \* Include robust error checking for file permissions, missing inputs, or invalid user actions.
- \* The script must be safe to run and must not execute any destructive commands (``rm -rf`` without strict boundaries, etc.).
- \* Since the project requires documenting all AI prompts, format your explanations clearly so I can understand the logic behind the code you generate.

## ## Your First Action:

Acknowledge these instructions. Confirm your understanding of the 6 phases and the 4-step execution strategy. Then, proceed immediately to **\*\*Step 1: Architecture & State Design\*\***, providing your detailed plan for state management, parallel processing methods, data extraction, and file organization. Do not write the final script yet.

Perfect. Here are the answers to your questions. Once you acknowledge these, please proceed immediately to Step 2 (Core Engine).

1. Tor entry point: Make ``proxychains`` (or ``proxychains4``) the primary mechanism. The project specification explicitly states: "Install Proxy Chains or Nipe for darknet access." Do not rely solely on the Tor daemon path.

2. Crawl depth: Only collect the newly-discovered links and append them to the ``index.txt`` file for the `*next*` invocation or a separate queue. Do not use recursive crawling (deep BFS) in the same run. We need the script to finish its core loop quickly and predictably for testing and demonstration purposes.

3. Politeness delay: Skip the delay entirely or keep it to an absolute minimum (e.g., max 1 second). We are explicitly maximizing CPU utilization and parallel processing for speed, so a long sleep jitter defeats the purpose.

4. UI mode: Use a single entry point. The script should open the Admin UI as the main interface, and the crawler should be launched in the background from within that UI. The user must be able to stay in the menu to check stats and logs while the crawler runs.

Proceed to Step 2: Core Engine.

Great work on the Core Engine. The logic for concurrent indexing and the ``flock`` implementation is spot on. Here are the requirements for Step 3 (Admin UI & Input):

1. Seed file format: Assume a clean format of one URL per line. If there are empty lines, the script should handle them gracefully (which the core engine already seems to do).
2. Keyword input: Use a sub-menu approach (Add/Remove one at a time). This is much more professional for an "Admin UI" and fits Requirement 4.2 better than a single prompt.
3. Background launch: Go with ``nohup ... &``. It's more resilient and ensures the crawl continues even if the UI menu is closed or the session hiccups.
4. UI Feedback: Ensure the Admin UI has a "Refresh" or "Auto-refresh" mechanism so the user can actually see the "Number of sites crawled" (Requirement 4.1.1) updating in real-time or upon request.

Please proceed to Step 3: Admin UI & Input. Fold the logic into a structure that prepares us for the final monolithic integration.

Excellent work on Step 3. The UI architecture, caching mechanism, and process control are perfect. Here are the answers for Step 4 (Integration):

1. Pre-flight check scope: Hard-fail and exit if ``proxychains`` or ``proxychains4`` is missing. The project spec explicitly states: "Install Proxy Chains or Nipe for darknet access." Without it, the core requirement cannot be met, so there's no point in a read-only mode for a first launch.
2. Tor connectivity test target: Stick with ``check.torproject.org``. It is fast, reliable, and perfectly validates that the proxy is tunneling traffic correctly, which satisfies Requirement 5.2 ("Validate the crawler's access to the darknet").
3. Single-instance enforcement: Yes, implement a UI lockfile (``$WORK_DIR/ui.pid``) so only one instance of the Admin UI can be running at a time. This prevents users from accidentally launching multiple background crawlers or getting confused with overlapping menu outputs.

Please proceed to Step 4: Integration & Polish.

Generate the final, monolithic script (``7736-39.s20.xe109.sh``). It must contain the academic header block exactly as specified, followed by the concatenated and properly sequenced contents of ``crawler_core.sh`` and ``crawler_ui.sh``. Make sure it's ready for immediate submission.



CRITICAL FIX & REFACTOR: The script must be fully autonomous and strictly under 1000 lines.

Issue 1: The current pre-flight checks (Phase A) hard-fail and ask the user to manually install tools or start the Tor service. This is a critical failure for the project requirements ("minimal human intervention").

Issue 2: The script is bloated (~1400 lines). A professional bash script of this scope should be tight and efficient, well under 1000 lines.

The user environment is a bare Kali Linux machine.

Update the monolithic script with the following strict constraints:

1. Autonomous Bootstrapping: Phase A MUST automatically handle its own dependencies. If ``tor`` or ``proxychains4`` are missing, or the network probe fails, the script must automatically self-heal by executing:

- ``sudo apt-get update``
- ``sudo apt-get install -y proxychains4 tor``
- ``sudo systemctl enable --now tor || sudo service tor start``
- Automatically uncomment ``proxy_dns`` and configure ``socks5 127.0.0.1 9050`` in ``/etc/proxychains4.conf`` using ``sed`` if needed.

It must do this autonomously without asking the user to run commands manually.

2. Code Golf & Minification (Under 1000 lines limit):

- Remove the ``run_selftest`` harness and its HTML fixtures entirely. It was useful for development, but we do not need it in the final academic submission.
- Remove the overly verbose ``# WHY`` comment blocks. Keep only brief, standard inline comments.
- Refactor repetitive UI printing blocks into tighter structures.

3. Execution: Output the ENTIRE updated monolithic script in a single code block so I can copy-paste the final version directly into ``7736-39.s20.xe109.sh``. Do not truncate.

CRITICAL FIX: The Tor network is definitely unblocked and working! A manual test with ``curl --socks5-hostname 127.0.0.1:9050 -I https://check.torproject.org/`` returned HTTP 200 OK after waiting 30 seconds for Tor to bootstrap.

The script's "Non-Active" status and ``[NO ACCESS]`` errors are purely because the script is impatient and does not wait for Tor to finish bootstrapping.

Update the monolithic script with these exact changes:

1. In Phase A, right after ensuring the ``tor`` service is running, add a MANDATORY ``sleep 35`` and a printed message "Waiting for Tor to bootstrap..." to give Tor the time it actually needs.
2. Increase the ``curl`` timeouts in ``verify_tor_connectivity`` and ``update_network_status_cache`` to ``--connect-timeout 45 --max-time 60``.
3. In ``fetch_url_via_proxy`` (the core worker), increase timeouts: ``--connect-timeout 45 --max-time 90`` to prevent false ``[NO ACCESS]`` logs for slow onion sites.
4. Maintain the rule: The script must be fully autonomous (auto-installs proxychains/tor if missing, auto-configures proxy\_dns) and strictly under 1000 lines (no self-test, keep comments brief).

Output the ENTIRE updated monolithic bash script in a single code block so I can copy-paste it perfectly.





